

Capitolo 24 — PROC-008 — Knowledge Integrity Assurance Loop

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-24
Data master	2026-08-30
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/24_proc_008_knowledge_integrity_assurance_loop.md
Source SHA	1fa089a
Release	2026-08-31T20:50:33.481Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

Parte VI — Il Libro dei Processi WCM Stato: FROZEN **Data:** 2026-08-30 **Scope:** WCM generale, domain-agnostic

24.0 Una memoria può contenere informazioni corrette e diventare comunque inaffidabile

Immaginiamo un sistema che possieda molti documenti corretti. Le decisioni sono state registrate. Gli stati sono stati aggiornati. Le fonti esistono. I processi sono documentati.

Sembra sufficiente.

Ma può accadere qualcosa di più sottile: una decisione cambia e un indice resta indietro; una relazione continua a puntare a un nodo ormai superseded; un ledger non viene aggiornato dopo un delta materiale; un entry point dichiara uno stato incompatibile con la fonte autorevole; un nodo importante diventa irraggiungibile dal percorso di navigazione previsto.

In questi casi il problema non è necessariamente che la conoscenza sia falsa. Il problema è che **la memoria organizzativa non è più affidabilmente utilizzabile come sistema.**

PROC-008 — Knowledge Integrity Assurance Loop esiste per questo.

La sua domanda fondamentale è:

la Persistent Organizational Memory è ancora coerente, navigabile, sufficientemente connessa e aggiornata rispetto all'ultimo cambiamento materiale?

E, se non lo è:

il drift può essere corretto senza interpretare il significato?

Questa seconda domanda è decisiva, perché separa la manutenzione meccanica dalla decisione semantica.

24.1 Che cos'è PROC-008

PROC-008 è il processo con cui WCM verifica la salute della memoria persistente dopo l'evoluzione del lavoro.

Il processo controlla aspetti come:

- coerenza dello stato;
- propagazione delle decisioni;
- validità delle relazioni;
- freshness dei ledger;
- nodi orfani;
- raggiungibilità delle fonti e degli indici;
- freshness del controllo rispetto all'ultimo delta materiale.

Se tutto è coerente, il loop può terminare senza lavoro cognitivo aggiuntivo.

Se emerge un'anomalia, il sistema deve prima classificarla.

Il principio è:

```
DETERMINISTIC FIRST
  ↓
ANOMALIA?
  ↓
ALLOWLISTED + DETERMINISTICAMENTE DIMOSTRABILE?
  ├── SÌ → CONTROLLED AUTO-REPAIR → RE-CHECK
  └── NO → NO WRITE → WISE / GATE APPLICABILE
```

Il punto non è riparare automaticamente il maggior numero possibile di cose. Il punto è automatizzare **soltanto ciò che possiede un unico risultato corretto dimostrabile senza interpretazione**.

24.2 PROC-008 non sostituisce PROC-006

Nel Capitolo 22 abbiamo visto **PROC-006 – Memory Consolidation & Consistency Loop**.

I due processi sono vicini, ma non equivalenti.

```
PROC-006
= consolida il delta e riallinea il patrimonio che quel delta impatta

PROC-008
= verifica che la memoria risultante sia integra, osservabile e affidabile
```

In termini semplici:

```
HO CAMBIATO QUALCOSA DI MATERIALE
  ↓
PROC-006
  ↓
HO PROPAGATO IL DELTA DOVE DOVEVA ARRIVARE?
  ↓
PROC-008
  ↓
LA MEMORIA RISULTANTE È DAVVERO SANA?
```

La distinzione evita due errori opposti.

Il primo è pensare che una write corretta garantisca automaticamente la coerenza globale.

Il secondo è trasformare l'assurance in un secondo processo di consolidamento che riscrive indiscriminatamente la memoria.

PROC-008 non deve “sistemare tutto”. Deve **misurare, classificare, riparare solo il meccanicamente dimostrabile e fermarsi davanti al significato**.

24.3 I trigger

Il processo può partire in tre modi principali.

Event-driven

È la modalità preferita dopo passaggi delicati o delta materiali, per esempio:

- decisione approvata o modificata;
- freeze, lock o promotion;
- cambio di stato o fase;
- nuovo output che crea dipendenze;
- nuovo capitolo, release o milestone;
- modifica di requisito, architettura o governance;
- aggiornamento di living ledger;
- supersession di un nodo rilevante.

Il concetto è semplice: **più un cambiamento può alterare relazioni e current-facing view, più è utile controllare l'integrità subito dopo**.

On-demand

Wise o il Board possono richiedere un controllo quando emergono dubbi, drift o incongruenze.

Periodico

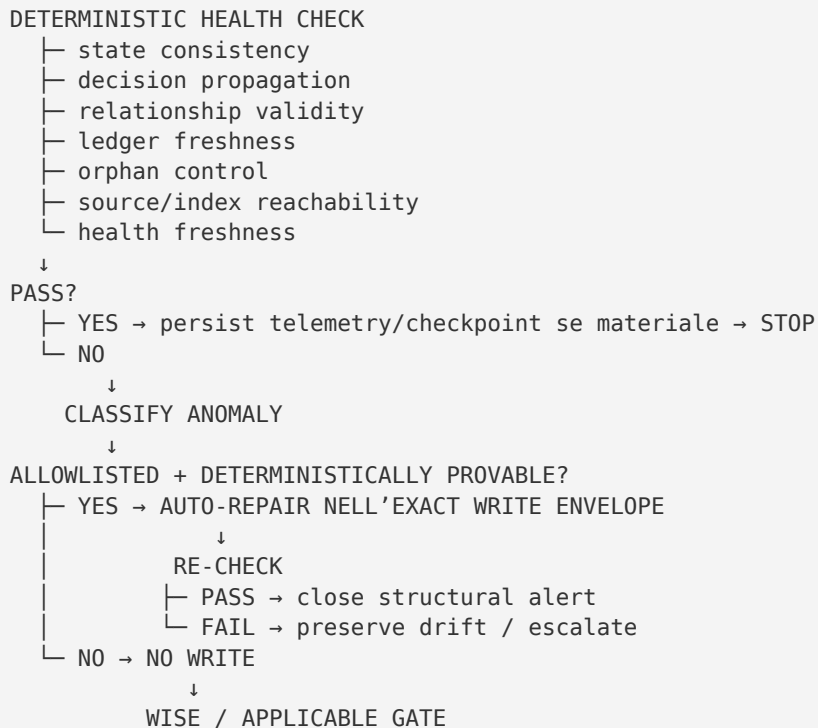
Un'assurance periodica leggera può intercettare omissioni non catturate dagli eventi.

La periodicità non sostituisce i check event-driven e non richiede automaticamente un heartbeat cognitivo LLM. Se i controlli deterministici sono verdi, il sistema non deve inventare lavoro cognitivo soltanto perché è arrivato un tick di scheduler.

24.4 Il flusso generale

Il processo canonico può essere letto così:

TRIGGER
↓



La sequenza contiene una regola essenziale:

un esito non-green non conferisce automaticamente authority di scrittura.

Trovare un problema e sapere come risolverlo sono due fatti diversi.

24.5 Deterministic Health Check

Il primo pass deve essere il più possibile deterministico.

Questo significa che il controllo deve basarsi su condizioni verificabili, non su un giudizio generico del tipo «sembra tutto coerente».

I check minimi comprendono sette aree.

1. State consistency

Gli entry point e i registri current-facing non devono dichiarare stati incompatibili con la fonte autorevole.

Questo non autorizza a scegliere arbitrariamente quale stato sia vero. Se esiste una gerarchia di source precedence, il checker applica quella gerarchia; se il conflitto non è risolvibile deterministicamente, l'anomalia viene escalata.

2. Decision propagation

Una decisione attiva o frozen deve essere raggiungibile e riflessa nei nodi che, per contratto, dipendono da essa.

Non significa che ogni decisione debba essere copiata ovunque. Significa che i punti che dichiarano di rappresentarla non possono restare silenziosamente indietro.

3. Relationship validity

Una relazione deve continuare a puntare a un target valido e semanticamente compatibile con il suo status.

Un target mancante può rendere la relazione **BROKEN**.

Una relazione potenzialmente invalidata da un delta, ma non ancora verificata, può diventare **AT_RISK**.

4. Ledger freshness

Un living ledger obbligatorio non può apparire current se è più vecchio dell'ultimo delta che lo riguarda.

Se non è stato verificato, deve essere esplicitamente trattato come stale o equivalente.

5. Orphan control

Un nodo materiale che, per contratto, dovrebbe avere relazioni significative non può diventare irraggiungibile senza che il problema venga osservato.

Non tutti i file senza link sono però automaticamente orphan.

6. Source/index reachability

Una fonte attiva rilevante deve essere raggiungibile dal percorso di navigazione previsto.

Un documento tecnicamente esistente ma invisibile agli entry point pertinenti può essere, di fatto, inutilizzabile per il bootstrap e il retrieval.

7. Health freshness

Questo è uno degli invarianti più importanti:

```
LAST_INTEGRITY_CHECK < LAST_MATERIAL_DELTA
↓
NOT HEALTHY
```

Un badge verde vecchio non resta verde per inerzia.

24.6 Knowledge Health: non è uno score cosmetico

PROC-008 usa stati di Knowledge Health che descrivono l'affidabilità corrente della memoria.

Stato	Significato
HEALTHY	check corrente, nessun drift critico, componenti entro le condizioni dichiarate
DEGRADED	anomalie note ma non bloccanti

STALE	il controllo non copre l'ultimo delta materiale o una current view non è stata verificata
CRITICAL	incoerenza che rende insicuro affidarsi alla memoria per bootstrap, authority o continuità
UNKNOWN	evidenza insufficiente per classificare

Un numero alto non può nascondere un blocker critico.

SCORE ALTO
+
BROKEN RELATION CRITICA
≠
HEALTHY

Lo score, quando esiste, è una sintesi. Gli invarianti e la severità restano più importanti.

24.7 Le sinapsi: più relazioni non significa memoria migliore

PROT-013 – Knowledge Synapse & Health Standard definisce una sinapsi come relazione tipizzata e intenzionale tra nodi persistenti.

Esempi del vocabolario generale includono:

DEPENDS_ON
DERIVED_FROM
IMPLEMENTS
CONSTRAINS
AFFECTS
SUPERSEDES
SUPERSEDED_BY
EVIDENCE_FOR
CONTRADICTS
RELATED_TO

Il valore non sta nel numero dei link.

Una relazione serve quando rende ricostruibile una dipendenza, un vincolo, una provenienza o un impatto reale.

Per questo il WCM applica un principio anti-gaming:

PIÙ SINAPSI
≠
KNOWLEDGE HEALTH MIGLIORE

Aggiungere collegamenti decorativi per “far sembrare più ricca” la rete peggiorerebbe il segnale invece di migliorarlo.

24.8 Controlled Auto-Repair

La parte più delicata di PROC-008 è l'auto-riparazione.

WCM non autorizza un agente a correggere liberamente la memoria ogni volta che un checker segnala un problema.

Una repair automatica è ammessa soltanto se esiste una **repair class attiva e allowlisted** con:

- ID e versione;
- precondition deterministiche;
- write scope esplicito;
- unica soluzione dimostrabile;
- re-check obbligatorio;
- evidence ricostruibile;
- rollback tramite Git history.

In forma compatta:

```
ANOMALIA
+
REPAIR CLASS ATTIVA
+
PRECONDITION VERIFICATE
+
UN SOLO RISULTATO CORRETTO DIMOSTRABILE
+
WRITE SCOPE ESATTO
=
AUTO-REPAIR ESEGUIBILE
```

Se manca uno di questi elementi, non esiste authority di auto-repair.

24.9 AR-001: un esempio di repair stretta

La baseline corrente include una repair class V1:

AR-001 – Authoritative State SHA Reconciliation.

Il suo scope è volutamente stretto.

Può aggiornare il fingerprint **expected_blob_sha** del contratto di Knowledge Health quando il mismatch SHA è **l'unico** failure di state consistency e le altre condizioni richieste risultano già verdi.

La repair non decide quale debba essere lo stato del progetto.

Non cambia il significato dello stato.

Non riscrive il contenuto per farlo combaciare con un'aspettativa.

Aggiorna soltanto il fingerprint con cui il contratto riconosce uno stato già correttamente propagato.

Questo esempio mostra perché l'auto-repair WCM è più simile a una correzione meccanica con guardrail che a una "AI che sistema da sola la knowledge base".

24.10 Il re-check è parte della repair

Una repair non è completa quando la write termina.

Deve seguire un nuovo controllo.

```
REPAIR APPLICATA
↓
RE-CHECK
├ PASS → alert strutturale chiudibile
└ FAIL → drift resta aperto / escalation
```

Questo impedisce di confondere l'esecuzione di una modifica con la dimostrazione che il problema sia realmente risolto.

L'idempotenza o la convergenza sono altrettanto importanti: ripetere il loop senza nuovi delta non deve continuare a produrre modifiche.

24.11 L'anti-pattern: riscrivere finché il checker diventa verde

Il processo vieta esplicitamente questa logica:

```
NON-GREEN
→ RISCRIVI FILE
→ RIPROVA
→ CONTINUA FINCHÉ IL CHECKER PASSA
```

Perché è pericolosa?

Perché trasforma il checker da strumento di verifica in obiettivo da ottimizzare.

Un sistema potrebbe produrre un punteggio perfetto distruggendo proprio l'informazione che avrebbe dovuto proteggere.

La health non va "massimizzata". Va resa **vera e spiegabile**.

24.12 Quando il Knowledge Steward può scrivere

Il Knowledge Steward opera entro un confine molto preciso.

Può eseguire autonomamente solo repair class attive e allowlisted.

Può inoltre mantenere elementi strutturali già determinati da authority esistente — per esempio telemetry, link o mirror — ma questo non significa che ogni manutenzione strutturale sia automaticamente auto-repair.

Per diventare repair automatica, una classe deve essere formalizzata con precondition e write scope espliciti.

La differenza è importante:

MANUTENZIONE POSSIBILE
≠
AUTO-REPAIR AUTORIZZATA

24.13 Quando deve fermarsi senza scrivere

Esistono anomalie che non devono essere risolte meccanicamente.

Il Knowledge Steward deve escalare senza scrivere quando, per esempio:

- due fonti autorevoli sono semanticamente incompatibili;
- serve scegliere fra interpretazioni diverse;
- la correzione cambierebbe canone, goal, roadmap o governance;
- una nuova relazione implica una decisione di significato;
- un broken target potrebbe essere stato rinominato, cancellato o superseded e il sistema non può provarlo;
- creare un ledger vuoto nasconderebbe knowledge mancante;
- è necessaria una riscrittura sostanziale per eliminare il drift.

Il principio è:

quando il significato non è dimostrabile deterministicamente, il loop smette di riparare e passa il problema all'authority appropriata.

24.14 BROKEN, AT_RISK, OPEN, SUPERSEDED

Le relazioni non sono semplicemente “presenti” o “assenti”.

Alcuni status aiutano a rappresentarne la condizione:

- **ACTIVE** — relazione corrente e verificata;
- **AT_RISK** — un delta potrebbe averla invalidata e serve verifica;
- **BROKEN** — il target o il contratto della relazione risultano rotti;
- **OPEN** — relazione ipotetica o non confermata;
- **SUPERSEDED** — relazione non più corrente ma preservata per lineage.

Una relazione **OPEN** non può essere trattata come fatto.

Una relazione **AT_RISK** non può essere promossa ad **ACTIVE** soltanto perché “sembra plausibile”.

Questi status permettono alla memoria di rappresentare anche l'incertezza senza cancellarla.

24.15 Orphan node: non ogni file isolato è un problema

Un nodo è orphan quando, **per la sua funzione e maturità**, dovrebbe possedere relazioni significative ma non ne possiede o non è raggiungibile dagli entry point pertinenti. La parte importante è “per la sua funzione e maturità”.

Un file temporaneo può non avere relazioni e non costituire alcun problema.

Una decisione frozen che dovrebbe vincolare più nodi ma non è raggiungibile da nessun indice rilevante è invece un segnale molto diverso.

L’orphan control deve quindi essere guidato da contract o regole dichiarate, non da una semplice conta dei backlink.

24.16 Freshness: il tempo conta, ma non come authority

Nei capitoli precedenti abbiamo visto che:

PIÙ RECENTE
≠
PIÙ AUTOREVOLE

PROC-008 aggiunge un’altra distinzione:

CHECK PIÙ RECENTE DEL DELTA
= condizione necessaria per dichiarare HEALTHY

La recency non rende una fonte autorevole. Ma la freshness del controllo è necessaria per sapere se la health dichiarata descrive ancora lo stato corrente.

Sono due concetti diversi che non vanno confusi.

24.17 Operational Heartbeat e Assurance Heartbeat

WCM separa due domande:

OPERATIONAL HEARTBEAT
= quale lavoro deve avanzare?

ASSURANCE HEARTBEAT
= la memoria risultante è ancora integra?

Possono condividere infrastruttura o scheduler, ma non devono condividere implicitamente lo stesso scopo.

Un Assurance Heartbeat non deve generare attività operative soltanto perché ha trovato la memoria sana.

Un Operational Heartbeat non sostituisce il controllo di Knowledge Health quando la next transition dipende da memoria affidabile.

La separazione evita che liveness, execution e assurance vengano compressi in un unico concetto ambiguo.

24.18 Sensitive work: quando la health può diventare un gate operativo

Se la Knowledge Health è **CRITICAL** o **STALE** e la transizione successiva dipende proprio dalla conoscenza non affidabile, il lavoro sensibile non dovrebbe proseguire.

Il flusso è:

```
CRITICAL / STALE
  ↓
NEXT TRANSITION KNOWLEDGE-SENSITIVE?
├─ NO → gestire secondo il contesto
└─ YES
  ↓
AUTO-REPAIR ALLOWLISTED DISPONIBILE?
├─ YES → REPAIR → VERIFY
└─ NO → RECONCILE / ESCALATE
  ↓
PASS?
├─ YES → CONTINUE
└─ NO → BLOCK / ESCALATE
```

Il failure di assurance non autorizza correzioni semantiche automatiche.

24.19 Output minimo: la health deve essere ricostruibile

Un check utile non può limitarsi a restituire un badge.

La baseline richiede che siano ricostruibili almeno elementi come:

```
project_id:
health_status:
checked_at:
last_material_delta_at:
last_reconciliation_at:
components:
  state_consistency:
  decision_propagation:
  relationship_validity:
  ledger_freshness:
  orphan_control:
metrics:
  active_synapses:
  new_synapses_since_checkpoint:
  modified_synapses_since_checkpoint:
  at_risk_synapses:
  broken_synapses:
  orphan_nodes:
  open_drifts:
checkpoint:
issues: []
```

Quando viene tentata una repair devono inoltre restare ricostruibili:

```
classification:
repairs_attempted:
repairs_applied:
files_changed:
escalations:
post_check_required:
```

Questo rende l'assurance osservabile e auditabile.

24.20 Mission Control e le projection non diventano source of truth

La health può essere mostrata in una UI con badge, metriche, componenti e issue. Ma la visualizzazione non acquisisce authority autonoma.

```
PERSISTENT HEALTH EVIDENCE / CONTRACT
↓
PROJECTION
↓
UI
```

Un badge verde non può prevalere su telemetria stale o su un blocker critico.

La UI deve aiutare l'utente a capire **perché** lo stato sia verde, degradato o critico, non semplicemente mostrarlo.

24.21 PROC-008 e WCM CHANGE

PROC-008 può produrre un esito Knowledge o Method Health verde anche durante un cambiamento materiale del WCM.

Ma:

```
KNOWLEDGE HEALTH GREEN
≠
WCM CHANGE CLOSED
```

Per un WCM CHANGE materiale, la closure richiede anche il processo di propagation e closure previsto da **PROC-012** e dal relativo standard.

PROC-008 verifica l'integrità della memoria.

Non sostituisce l'authority, non sostituisce la propagazione completa e non chiude autonomamente il change.

24.22 Failure mode principali

I failure mode più importanti sono:

Falso verde

La health viene dichiarata **HEALTHY** anche se il check è precedente all'ultimo delta materiale.

Auto-repair semantica

Il sistema modifica contenuto, canone o relazioni interpretative senza una soluzione deterministica unica.

Score gaming

Si aggiungono link o si alterano artefatti per migliorare una metrica anziché riflettere la realtà.

Repair senza re-check

La write viene considerata prova sufficiente della risoluzione.

Broken target risolto per intuizione

Il sistema indovina che un file sia stato rinominato o superseded senza evidence deterministica.

Orphan inflation

Tutti i file senza backlink vengono classificati come orphan, ignorando funzione e maturità.

Heartbeat confusion

Assurance e avanzamento operativo vengono trattati come lo stesso loop.

UI-as-truth

La projection verde viene considerata più affidabile della persistent evidence che la alimenta.

24.23 Relazioni con gli altri processi

PROC-008 vive dentro una rete di processi e protocolli.

Relazione	Funzione
PROC-006	consolida il delta che l'assurance deve poi verificare
PROT-013	definisce sinapsi, health invariants e boundary di auto-repair

PROC-011	riconcilia deterministicamente lo stato esecutivo quando il problema è state drift strutturato
PROC-012	usa anche Method/Knowledge Health nel closure gate dei WCM CHANGE materiali
PROT-019	impedisce di confondere health verde con closure completa del change

Una distinzione particolarmente importante riguarda **PROC-011**.

Se il problema è una state reconciliation meccanica ricostruibile dal runtime strutturato, il percorso corretto non è inventare una decisione cognitiva di knowledge repair: si applica il processo deterministico di state reconciliation.

24.24 Maturity: cosa possiamo affermare oggi

Il Process Register corrente qualifica PROC-008 come:

VALIDATED BY GOVERNANCE / AUTO-REPAIR V1 FIELD VALIDATION.

Questa formulazione va letta con precisione.

Significa che:

- il processo e i suoi confini sono parte della baseline governata;
- esiste una baseline di Controlled Auto-Repair V1;
- il modello è in field validation nel perimetro dichiarato;
- non viene affermata una validazione universale su qualsiasi dominio, repository, organizzazione o classe di drift.

In particolare, la baseline corrente non autorizza semantic auto-repair generico.

24.25 La lezione del processo

La lezione più importante di PROC-008 può essere riassunta così:

una memoria organizzativa non è affidabile soltanto perché contiene documenti corretti; deve anche mantenere coerenti relazioni, current view, freshness e provenance.

Ma il secondo principio è altrettanto importante:

integrità non significa libertà di riscrittura.

WCM cerca di automatizzare il controllo e le riparazioni strettamente meccaniche, lasciando fuori dal perimetro automatico ciò che richiede interpretazione, authority o modifica di significato.

È questa separazione che trasforma l'assurance da "Al che sistema la knowledge base" in un **immune loop controllato**.

Source Map

Fonti canoniche utilizzate per il Technical Truth Pass:

- `WCM_AGENT_START.md`;
- `wcm/documentation/process-memory-book/BOOK_INDEX.md`;
- `wcm/process-book/processes/PROC-008_KNOWLEDGE_INTEGRITY_ASSURANCE_LOOP.md`;
- `wcm/process-book/protocols/PROT-013_KNOWLEDGE_SYNAPSE_HEALTH_STANDARD.md`;
- `wcm/process-book/PROCESS_REGISTER.md` per la maturity corrente.

Maturity qualifier

Questo capitolo descrive la baseline corrente di **PROC-008** senza estenderne l'authority. **VALIDATED BY GOVERNANCE / AUTO-REPAIR V1 FIELD VALIDATION** non equivale a field validation universale. Le auto-repair restano limitate alle repair class attive, allowlisted e deterministicamente provabili; i conflitti semantici restano fuori dal write scope automatico.